1주차 3일차: 입력과 버튼 이벤트 — 30분

1. 학습 목표 — 2분

- Click 이벤트와 C# 메서드를 연결합니다.
- C#에서 입력 컨트롤의 값을 읽고 결과 컨트롤에 씁니다.
- 버튼을 누를 때마다 동작하는 프로그램을 실행합니다.

시작 상태는 2일차 완료본입니다.

2. 핵심 개념 설명 — 6분

이벤트는 "어떤 일이 일어났다"는 알림입니다. 버튼의 Click 이벤트에 메서드를 연결하면 사용자가 버튼을 누를 때 WPF가 그 메서드를 호출합니다. 이 메서드를 이벤트 처리기라고 부릅니다.

콘솔 앱에서 입력문을 차례로 실행하던 방식과 달리, 창은 열린 상태로 사용자의 동작을 기다립니다. 클릭할 때마다 처리기가 다시 호출됩니다.

오늘의 연결은 다음과 같습니다.

```
버튼 클릭
  → XAML의 Click="RunButton_Click"
  → C#의 RunButton_Click 호출
  → InputTextBox.Text 읽기
  → ResultTextBlock.Text에 쓰기
```

InputTextBox와 ResultTextBlock은 XAML의 x:Name으로 지정한 이름입니다. 초기화가 끝나면 Code-behind에서 그 이름으로 화면 요소에 접근할 수 있습니다. TextBox에 입력된 글자는 Text라는 string 속성에 들어 있습니다.

object sender는 이벤트를 일으킨 객체, RoutedEventArgs e는 이벤트 관련 정보입니다. 오늘은 사용하지 않으므로 인자 두 개를 그대로 둡니다. 이벤트가 화면 안에서 전달되는 자세한 원리는 다루지 않습니다.

실무·면접 기초: Code-behind는 화면 객체를 직접 읽고 수정하기 편합니다. 데이터와 동작이 늘어날 때의 역할 분리는 바인딩과 MVVM을 배운 뒤 비교합니다.

3. 짧고 이해하기 쉬운 코드 예제 — 5분

2일차에서 바뀌는 핵심은 XAML의 Click="RunButton_Click"과 C#의 메서드 한 개입니다. 위치가 헷갈리지 않도록 두 파일 전체를 제공합니다.

아래는 오늘 완료 시점의 두 파일 전체입니다. 같은 이름의 파일 내용을 각각 교체합니다. 두 파일을 하나로 합치지 않습니다.

MainWindow.xaml

```
<Window x:Class="WpfTestApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="WPF 텍스트 테스트" Height="360" Width="640"
        MinHeight="300" MinWidth="420">
    <Grid Margin="16">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto" />
            <RowDefinition Height="Auto" />
            <RowDefinition Height="*" />
        </Grid.RowDefinitions>

        <TextBlock Text="테스트할 텍스트를 입력하세요."
                   Margin="0,0,0,8" />

        <Grid Grid.Row="1">
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="*" />
                <ColumnDefinition Width="Auto" />
            </Grid.ColumnDefinitions>

            <TextBox x:Name="InputTextBox"
                     Text="안녕하세요, WPF"
                     VerticalContentAlignment="Center"
                     Margin="0,0,8,0" />
            <Button Grid.Column="1" Content="실행"
                    Padding="16,6"
                    Click="RunButton_Click" />
        </Grid>

        <StackPanel Grid.Row="2" Margin="0,16,0,0">
            <TextBlock Text="결과" Margin="0,0,0,8" />
            <TextBlock x:Name="ResultTextBlock"
                       Text="아직 실행하지 않았습니다."
                       TextWrapping="Wrap" />
        </StackPanel>
    </Grid>
</Window>
```

MainWindow.xaml.cs

```
using System.Windows;

namespace WpfTestApp;

public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
    }
```

```
    private void RunButton_Click(object sender, RoutedEventArgs e)
    {
        ResultTextBlock.Text = InputTextBox.Text;
    }
}
```

ResultTextBlock.Text = InputTextBox.Text;는 실행되는 순간의 입력값을 읽어서 복사하는 일반적인 C# 대입문입니다. 자동으로 계속 연결되는 기능은 아닙니다. 버튼을 누른 뒤 입력창을 다시 편집해도 결과는 다음 클릭까지 그대로입니다.

4. Visual Studio에서 따라 하는 실습 — 14분

1. **2분:** 실행을 종료하고 MainWindow.xaml과 MainWindow.xaml.cs를 엽니다.
2. **5분:** 두 파일을 위 예제 전체로 교체하고 저장합니다. XAML의 Click 이름과 C# 메서드 이름이 정확히 같은지 확인합니다. 이벤트를 자동 생성하는 메뉴는 사용하지 않아도 됩니다.
3. **3분:** F5를 누릅니다. 기본 텍스트가 입력된 상태에서 실행 버튼을 누르면 결과에 같은 글자가 나타나는지 봅니다.
4. **2분:** 입력을 "두 번째 입력"으로 바꿉니다. 버튼을 누르기 전에는 기존 결과가 유지되고, 버튼을 누른 뒤에 바뀌는 것을 확인합니다.
5. **2분:** 창을 닫고 C#의 대입문을 다시 봅니다. 왼쪽은 결과를 쓸 대상, 오른쪽은 값을 읽을 대상입니다.

실행 버튼을 눌러도 반응하지 않으면 XAML에 Click이 들어 있는지 확인합니다. 빌드 오류가 나면 두 파일의 처리기 이름과 x:Name 철자를 확인합니다.

5. 이번 학습에서 반드시 이해해야 할 핵심 내용 요약 — 3분

- 이벤트 처리기는 사용자의 동작에 따라 WPF가 호출하는 C# 메서드입니다.
- XAML의 Click과 C#의 메서드 이름으로 연결합니다.
- x:Name이 있으면 Code-behind에서 해당 요소를 참조할 수 있습니다.
- 오늘은 클릭할 때만 값을 복사합니다. 데이터 바인딩은 아직 사용하지 않았습니다.

완료본은 day03 폴더에 있습니다. 다음은 4일차: 입력 → 실행 → 결과 완성입니다.